Il modello di Von Neumann

Il modello di Von Neumann

È un modello astrato su cui si basano tutti i dispositivi elettronici moderni.
Questo modello si basava principalmente su una 4 dispositivi che stanno dentro all'elaboratore:

1. CPU
2. RAM
3. Dispositivi I/O
4. Bus di sistema

Bus di sistema

connette la CPU, RAM e le periferiche di I/O a gli altri componenti del sistema.
Permette di:
- **trasferire i dati**
- **Trasferire le istruzioni**
- **Trasferire i segnali di controllo tra i componenti, permettendo di interagire tra loro e collaborare.**

A sua volta il bus di sistema è suddiviso i **3 categorie:**

1. **Bus Dati:**
   è utilizzato per il trasferimento dei dati tra i vari componenti del sistema, come CPU, memoria e dispositivi di I/O.
   La sua larghezza (es. 8, 16, 32, 64 bit) determina la quantità di dati che possono essere trasferiti contemporaneamente.
2. **Bus di controllo:**
   trasporta segnali di controllo e sincronizzazione tra i componenti, come i segnali di lettura/scrittura, conferme di trasferimento dati, o segnali di interrupt. Gestisce il coordinamento delle operazioni.
3. **Bus di Indirizzi:**
   serve per specificare l'indirizzo di memoria o di I/O a cui la CPU vuole accedere.
   La sua larghezza determina il numero massimo di locazioni indirizzabili

(es. un bus di indirizzo a 32 bit può indirizzare fino a 2^{32} locazioni di memoria).

CPU (Central Processing Unit)

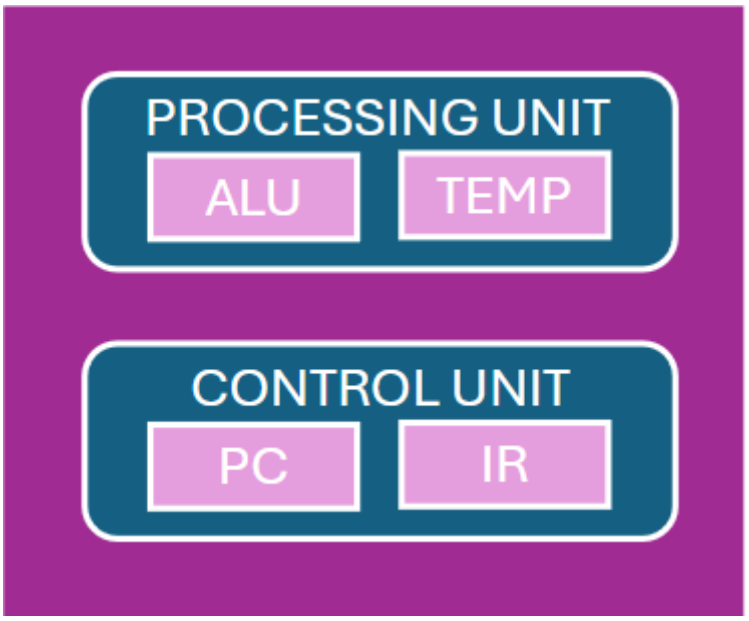
Central Processing Unit (CPU) è il principale componente di elaborazione del computer. In termini semplici, viene spesso chiamata il "cervello" del sistema, poiché:

- **esegue le istruzioni dei programmi** .
- **svolge calcoli** .
- **gestisce il flusso di dati tra memoria e periferiche** .
  Al suo interno sono presenti due unità, cioè

1. **PU (Processing Unit):**
   L'unità di elaborazione è una parte centrale della CPU responsabile

   1. **dell'esecuzione delle istruzioni**
   2. **della gestione delle attività di elaborazione dei dati.**

   Include diversi componenti chiave che lavorano insieme per eseguire:
   1. **operazioni aritmetiche e logiche,**
   2. **controllare il flusso di istruzioni**
   3. **memorizzare i risultati intermedi.**
2. **CU (Control Unit):**
   L'unità di controllo è una parte cruciale della CPU che **dirige il funzionamento del processore.**
   1. **Interpreta le istruzioni del programma**
   2. **genera segnali di controllo**
   3. **coordina le attività degli altri componenti della CPU**
   4. **assicura che le istruzioni vengano eseguite nella sequenza e nei tempi corretti.**



Inside The CPU.png

Processing Unit (PU)

Come possiamo vedere dall'immagine sopra, la PU è a sua volta suddivisa in altri 2 unità:

1. **ALU (Arithmetic Logic Unit):**
   L'ALU è un componente critico della CPU che esegue operazioni aritmetiche (come addizione, sottrazione, moltiplicazione e divisione) e operazioni logiche (come AND, OR, NOT e XOR).
   È essenziale per eseguire calcoli e prendere decisioni basate su confronti logici.
2. TEMP (**Temporary Register**):
   I registri temporanei (TEMP) sono posizioni di archiviazione piccole e veloci all'interno della CPU utilizzate per contenere dati e risultati intermedi durante l'esecuzione delle istruzioni. Forniscono un accesso rapido ai valori utilizzati di frequente e aiutano a ottimizzare la velocità di elaborazione riducendo la necessità di accedere alla memoria principale più lenta.

Control Unit (CU):

Come per la PU, anche la CU è suddivisa in altre 2 unità:

1. **PC (Program Counter):**
   Il Program Counter (PC) è un registro all'interno dell'unità di controllo che contiene l'indirizzo dell'istruzione successiva da eseguire.
   Incrementa dopo che ogni istruzione è stata recuperata, garantendo un flusso

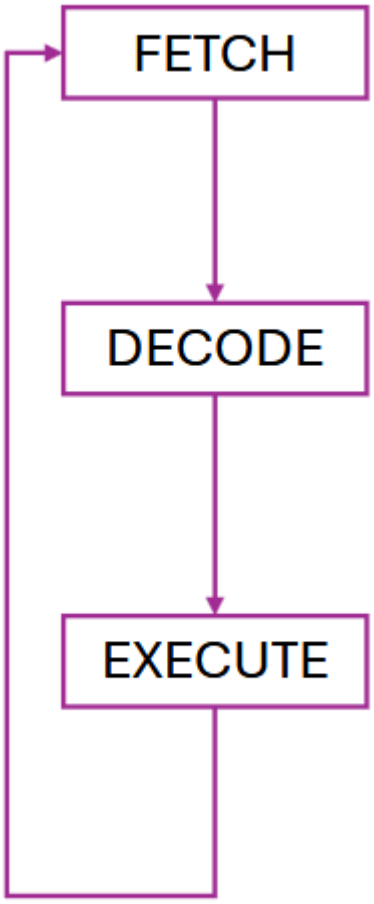
sequenziale di esecuzione del programma, a meno che un'istruzione di salto o di ramo non alteri il flusso.

2. **IR(Instruction Register):**
   Il registro delle istruzioni (IR) è un componente dell'unità di controllo che contiene temporaneamente l'istruzione corrente in esecuzione.
   Memorizza l'istruzione recuperata dalla memoria, consentendo all'unità di controllo di decodificarla ed elaborarla, indirizzando le azioni necessarie ad altre parti della CPU.

Ciclo di vita dell'istruzione

Si suddivide in 3 step che si ripetono di continuo:

1. Fetch:
   La CPU recupera l'istruzione successiva dalla memoria, come indicato dal Program Counter (PC).
   L'istruzione viene caricata nel Instruction Register (IR).
2. Decode:
   La CU interpreta l'istruzione recuperata nell'IR.
   Determina l'operazione da eseguire e identifica gli operandi necessari.
3. Execute:
   La CPU esegue l'operazione specificata dall'istruzione.
   Ciò può comportare operazioni aritmetiche o logiche eseguite dal ALU, trasferimento di dati o operazioni di controllo.



Instruction Cycle.pdf.png

RAM

Random Access Memory (RAM):
memoria casuale ad accesso rapido e volatile: cioè una volta che viene spento il computer la RAM viene svuotata.

Glossario

- **Casuale (Random Access):** **significa che la CPU può accedere direttamente a qualsiasi cella di memoria conoscendone l'indirizzo, senza dover scorrere sequenzialmente tutte le altre.**
- **Rapida:** **perché molto più veloce rispetto a memorie di massa (es. disco rigido)**.
- **Volatile:** **il contenuto viene perso allo spegnimento del computer**.

Ruolo nell'architettura di Von Neumann

Nell'architettura di Von Neumann, la memoria è tipicamente Riferita alla **RAM (Random Access Memory)**, che è un componente critico che **memorizza temporaneamente sia i dati che le istruzioni del programma mentre il computer è in esecuzione.**
Quando un dispositivo è operativo, sia i programmi (Es: Word) che i dati associati (il documento di Word che stai scrivendo) sono archiviati in memoria.

Operazioni principali sulla memoria

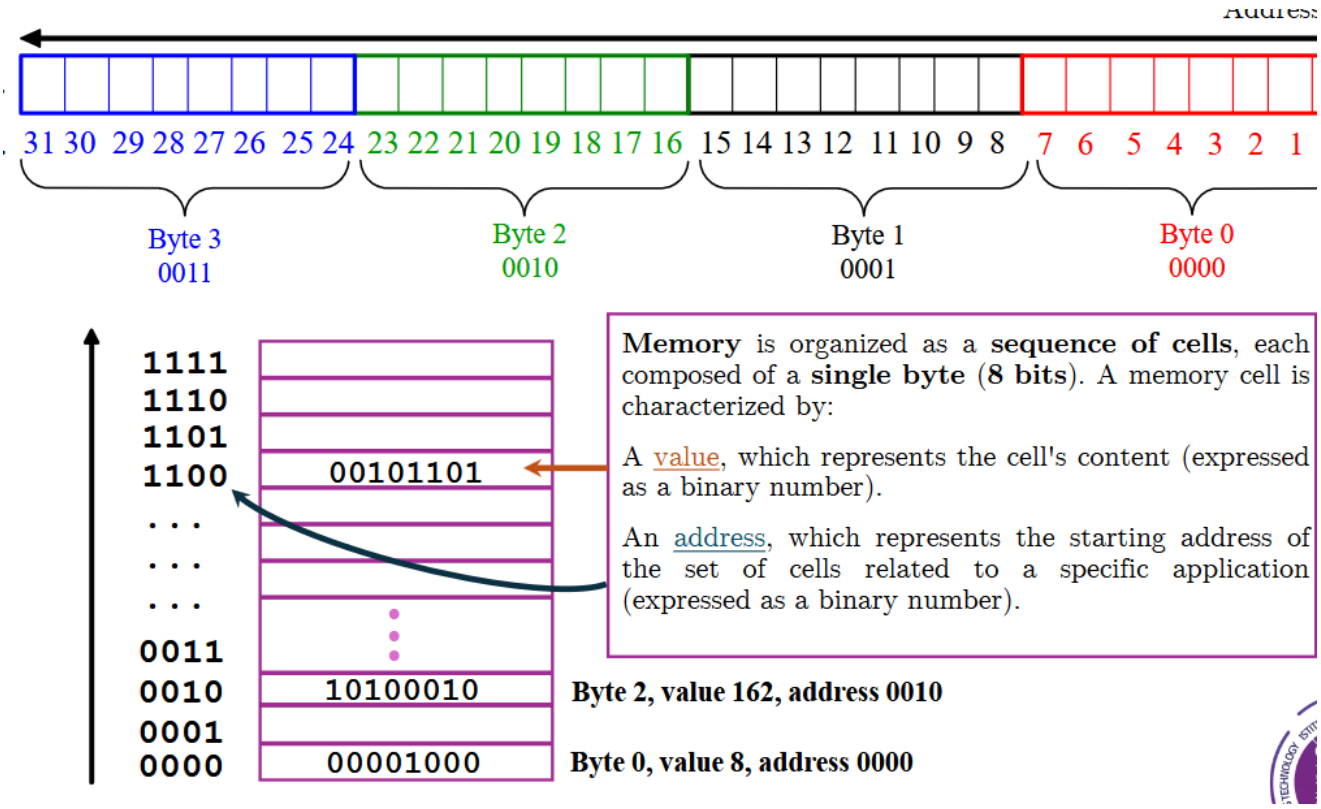
Sulla memoria possiamo compiere 2 esecuzioni:

1. Fetch:
   Operazione di **[[#**Operazione di lettura (FETCH) **– leggere un dato dalla posizione di memoria** A**|lettura]]:**
   - **in cui la CPU recupera un'istruzione o un dato dalla RAM per elaborarlo.**
   - In altre parole legge un dato o un'istruzione da una cella della RAM.
   - Esempio: prende l'istruzione `SOMMA` da eseguire.
2. Store:
   Operazione di **[[#**Operazione di scrittura (STORE) **– scrivere un dato** X **nella posizione di memoria** A**|scrittura]]:**
   in cui la CPU memorizza un dato nella RAM per un uso futuro.
   In altre parole la CPU scrive un dato in una cella della RAM.
   Esempio: salva il risultato della somma nella RAM.

Queste operazioni avvengono attraverso un sistema di indirizzamento chiamato **memory addressing:**
che consente alla CPU di individuare e gestire le informazioni nella RAM.

Organizzazione della memoria

La memoria è una sequenza di celle, ognuna di 8 bit (1 byte).



Memory Addresses.png

Ogni **cella** di memoria è caratterizzata da:

- **Valore: il contenuto della cella (un numero binario, es. `1010010`)**.
- **Indirizzo: la posizione univoca della cella nella memoria (espresso anch'esso in binario, es. `0010`)**.

Esempio con lo schema

- Byte 0 → indirizzo `0000`, valore `00001000` (= 8 in decimale).
- Byte 2 → indirizzo `0010`, valore `10100010` (= 162 in decimale).

MDR e il MAR Register

All'interno della **CPU**, due registri fondamentali per la comunicazione con la memoria sono:

- il **MDR (Memory Data Register)**
- il **MAR (Memory Address Register)**

Entrambi appartengono all'**Unità di Controllo (CU)** e lavorano sempre insieme.

1. **MDR:**
   è un registro che contiene i dati che sono stati recuperati dalla memoria o che devono andare in memoria, quindi che vanno dalla CPU alla RAM e viceversa.

Quando i dati vengono letti dalla memoria, vengono prima caricati nell'MDR prima di essere elaborati dalla CPU.
Allo stesso modo, **quando i dati vengono scritti in memoria, vengono inseriti nell'MDR prima di essere memorizzati nella posizione di memoria specificata.**
In sostanza, il MDR agisce come un buffer:

- se leggo dalla RAM → i dati passano prima dal MDR, poi alla CPU;
- se scrivo in RAM → i dati passano dalla CPU al MDR, e poi vanno in memoria.
- Questo evita irregolarità e rende fluido lo scambio dati.
  In sostanza facilita il trasferimento regolare dei dati tra CPU e memoria.
  In sintesi: il MDR dice *"Quali dati sto portando o ricevendo dalla memoria?"*

2. **MAR:**
   contiene gli indirizzi di memoria dove i dati devono essere scritti o recuperati fisicamente, si trova nella CPU ma permettono l'interazione tra la RAM e la CPU.
   Il Memory Address Register (MAR):
   è un registro all'interno della CPU che contiene l'indirizzo della posizione di memoria a cui accedere successivamente.
   Durante la fase di recupero (fetch) del ciclo di istruzioni, il MAR contiene l'indirizzo dell'istruzione da recuperare (fetched).
   Durante le operazioni sui dati, il MAR conserva l'indirizzo dei dati da leggere o scrivere in memoria.
   Il MAR garantisce l'accesso alla corretta posizione di memoria, consentendo il recupero e l'archiviazione precisi dei dati.
   In sintesi: il MAR dice *"Dove devo andare in memoria?"*

```
^280869
 ^a2aca9
```

Relazione tra MAR e MDR

Quando la CPU deve leggere un dato dalla memoria:

1. L'indirizzo della cella da leggere → viene caricato nel **MAR**.
2. L'istruzione o il dato contenuto in quella cella → viene copiato nel **MDR**.
3. Il contenuto del MDR → viene usato dalla CPU per l'elaborazione.

☰ Schema semplificato della lettura

```
CPU
├── MAR → contiene indirizzo da leggere
├── MDR ← riceve dati dalla RAM
└── CU  → coordina lo scambio con la RAM
```

Quando la CPU deve scrivere in memoria:

1. L'indirizzo di destinazione → va nel **MAR**.
2. I dati da scrivere → vanno nel **MDR**.
3. Il contenuto del MDR → viene trasferito nella cella di memoria puntata dal MAR.

☰ Schema semplificato della scrittura

```
CPU
├── MAR → contiene indirizzo di destinazione
├── MDR → invia dati alla RAM
└── CU  → coordina lo scambio con la RAM
```

Entrambi questi registri sono gli indirizzi dove vengono eseguiti le operazioni di fetch (MDR) e lo store (MAR).

Le operazioni di fetch e store

Il funzionamento del modulo di memoria può essere descritto attraverso due operazioni fondamentali: **lettura (fetch)** e **scrittura (store)**.

Operazione di lettura (FETCH) – leggere un dato dalla posizione di memoria A:

1. **L'indirizzo della cella desiderata (A) viene caricato nel MAR (Memory Address Register).**
2. **Viene inviato un segnale di lettura alla memoria.**

3. **Il contenuto della cella viene trasferito nel MDR (Memory Data Register), da cui potrà essere utilizzato dalla CPU.**

Scenario

Immaginiamo la memoria come un grande schedario, dove ogni cassetto è una cella identificata da un **indirizzo**.
La CPU vuole leggere cosa c'è nel cassetto n. `100`:

- L'indirizzo `100` viene messo nel **MAR** (come scrivere su un post-it: "apri il cassetto `100`").
- La CPU invia il comando di **lettura**.
- Supponiamo che nella cella `100` ci sia il valore `7`: questo viene copiato nel **MDR**.
- Ora la CPU può usare il valore `7` letto dalla memoria.

> **Important**
>
> In pratica: **la CPU legge il valore `7` dalla cella `100` e può riutilizzarlo nelle sue operazioni**.

Operazione di scrittura (STORE) – scrivere un dato X nella posizione di memoria A:

1. **Il dato da scrivere (X) viene caricato nel MDR.**
2. **L'indirizzo della cella di destinazione (A) viene caricato nel MAR.**
3. **Viene inviato un segnale di scrittura alla memoria, che memorizza il valore X nella cella indicata.**

Scenario

Continuando l'analogia con lo schedario:

- La CPU vuole scrivere il valore `42` nel cassetto n. `200`.
- Il valore `42` viene caricato nel **MDR**.
- L'indirizzo `200` viene inserito nel **MAR**.
- La CPU invia il comando di **scrittura**.
- La memoria prende il valore `42` e lo inserisce nella cella `200`.

Important

In pratica: **la CPU scrive il valore `42` nella cella `200`**.

Analogia con Python

Per capire meglio come lavorano i registri **MAR** e **MDR**, e come funzionano le operazioni di *fetch* (lettura) e *store* (scrittura), possiamo fare un parallelo con le variabili di Python.

In Python, una variabile non è un contenitore fisico del dato ma un **riferimento** a un oggetto presente in memoria.
Quando scriviamo ad esempio:

```
x : int = 52
```

La variabile `x` diventa un **nome** che fa riferimento all'oggetto `52` memorizzato in un'area della RAM gestita dall'interprete Python.

In maniera simile, nella CPU il **MAR non contiene il dato stesso, ma l' "indirizzo" della cella di memoria dove quel dato si trova, mentre il MDR contiene il valore letto o da scrivere**.

Caso lettura (read/fetch)

1. **La CPU deve leggere il valore di `x`.**
2. **L'indirizzo di memoria dove si trova `52` → viene messo nel MAR.**
3. **Il contenuto di quella cella (cioè `52`) → viene copiato nel MDR.**
4. **La CPU legge l'MDR e usa il valore (`52`) per i suoi calcoli.**

Caso scrittura (write/store)

```
x:int = 99
```

Adesso si vuole salvare il valore `99` al posto del `52`.

1. **La CPU decide dove scrivere: l'indirizzo della cella → caricato nel MAR.**
2. **Il valore da scrivere (`99`) → messo nel MDR.**

3. **L'Unità di Controllo ordina di scrivere: il contenuto dell'MDR viene trasferito nella cella di memoria il cui indirizzo è nel MAR.**
4. **Ora nella cella che prima conteneva** `52` **c'è** `99`**.**

Device I/O

Le operazioni di **input/output (I/O)** sono essenziali per permettere a un sistema informatico di comunicare con l'ambiente esterno attraverso i dispositivi periferici.

Input

- **Obiettivo: acquisire dati dall'esterno e fornirli al computer per l'elaborazione.**
- **Dispositivi tipici: tastiera, mouse, scanner, microfono, fotocamera.**
- **Funzionamento: i dispositivi di input traducono azioni dell'utente o dati provenienti dall'ambiente in segnali digitali, interpretabili ed elaborabili dal sistema.**

Output

- **Obiettivo: restituire all'esterno i dati elaborati dal computer in una forma comprensibile.**
- **Dispositivi tipici: monitor, stampante, altoparlanti, proiettore.**
- **Funzionamento: i dispositivi di output trasformano i segnali digitali provenienti dal sistema in una forma percepibile dall'uomo: testo, immagini, suoni o risultati fisici.**